

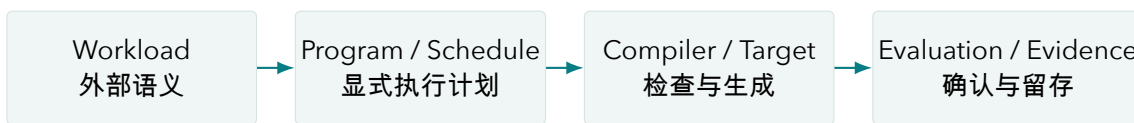
研究报告 / 技术体系与实验证据

跨硬件优化机制复用 与编译器协同演进

Cross-Hardware Optimization Reuse and Compiler Co-Evolution

秦海岩 Haiyan Qin
北京航空航天大学

2026 年 10 月 6 日
MoE 案例增补: 9 月 28 日
协同演进动机补充: 10 月 2 日
历史实现快照 / 源码 677eb444



报告边界 主体章节所读源码固定为 677eb444。第8.8节是 2026 年 9 月 28 日的 MoE 案例增补，其实验代码固定于尚未合入主线的 2c638aad；该次增补只纳入案例记录，不将后续代码追认为主体快照。2026 年 10 月 6 日更新研究定位与第2节共同实验约定。历史实现和实验数字仍绑定原来源；新增研究设计不改变它们的验收范围。

摘要

高性能实现需要目标特化；一次优化中形成的数据流改写、适用条件与反例，则可能帮助后续任务减少重复搜索。open-cake-ir 以 Compiler 为核心，用带类型的 Schedule 和 Program 记录执行选择，将经过审查的发现整理为显式变换、目标诊断或编译能力。Agent 在目标侧重新选择分块、工作分配和资源配置；独立评测确认设备正确性与性能。原生工程探索也可提供新发现，相关能力在后继提交与新任务中重新验证。

本报告分别讨论表示与诊断、额外材料与变换权限、lowering 自由度和系统演进的作用。原生 DSL 加等效工具、同动作集合的非 LLM 搜索及来源知识对照，是后续研究的不同入口。实现基础和局部收益不能直接替代跨目标搜索成本的因果证据。

历史实现章节保留源码 677eb444 的八个目标、179 项语料预期及各自设备案例；TinyGEMM2 的性能差距与 MoE 的有界正确性也维持原结论。2026 年 10 月的研究更新统一各硬件分支的问题、对照和成本口径，已有 Run 保持原有源码、预算和接受规则。

关键词：GPU 内核；条件化改写；目标适配；智能体；编译器；经验复用

ABSTRACT

open-cake-ir studies conditional optimization reuse across hardware, with the Compiler as its core. Agents author explicit execution plans; reviewed discoveries from structured or qualified native exploration may become transformations, diagnostics or lowering capabilities. Target decisions are reselected and independently evaluated. Representation effects, executable-tool effects, lowering freedom and system-successor effects require distinct controls. The October research update defines a common agenda for all hardware branches. Historical implementation chapters retain source 677eb444, its 179 corpus expectations and the original scope of each device result. Controlled transfer benefits remain a research question rather than a consequence of the architecture.

阅读约定

阅读结果时，请先看它经过哪一层检查。语料门说明编译器符合已审查的软件预期；设备正确性与计时结果还要说明目标型号、任务、运行版本和测量质量。比较两种智能体编写环境的效果，则需要预先分组的独立运行和同一套确认规则。本文把历史实验、当前仓库记录与 CAKE 论文报告的数字分别标明。

内容导览

1	问题、研究对象与论点	3	7	实验方法与证据等级	14
1.1	背景：公式、执行计划与智能体反馈	3	7.1	一次 Run 的四阶段	14
1.2	变化中的工具链与编译能力	3	7.2	双时间尺度与 Finding 的负责位置	14
1.3	CAKE 论文提供的研究起点	3	7.3	研究对照的单位和参考访问	15
1.4	本工作的定位与证据范围	4	7.4	证据保存、回放与结论资格	15
			7.5	结论阶梯	15
2	共同研究主题与跨平台实验	4	8	实现覆盖与代表性设备案例	15
2.1	从优化发现到条件化机制	4	8.1	平台覆盖是分层事实	15
2.2	相关路线与增量贡献	4	8.2	NVIDIA：任务族覆盖与外部比较	17
2.3	独立原生发现与后继晋升	5	8.3	Apple、Hygon、AMD：不同结论等级	17
2.4	各硬件分支回答同一主问题	5	8.4	MetaX：单内核、完整输出和实际 Run 分别验收	17
3	设计原则与实现边界	6	8.5	案例 A：C550 的局部 tiling 收益	18
4	系统合同与职责边界	6	8.6	案例 B：B300 的对齐条件与质量门	18
4.1	从任务到可审计结论	6	8.7	两个局部案例的结论范围	18
4.2	编译器：检查程序，再生成目标源码	7	8.8	案例 C：B300 MoE 的事件流水与 native CUDA lowering	18
4.3	实验系统：固定一次优化的条件	7	8.9	历史 B200 案例的放置方式	19
4.4	评测与证据：先判对，再解释速度	7	9	与 CAKE 对照：相同的问题，不同的证据	19
4.5	两个时钟：候选搜索与编译器演进	8	9.1	TinyGEMM2：相同任务上的性能差距	20
5	程序表示：任务、完整程序与执行计划	8	10	跨硬件复用：从一次改写到可检验的收益	21
5.1	显式的计划，而非新的布局代数	8	10.1	可复用的是机制，参数要在目标卡 重选	21
5.2	受限 Python 前端与同一规范 IR	9	10.2	E/P 四组实验	21
5.3	从 Python Schedule 到 Triton：一次实 际生成	9	10.3	从 B300 到 Hygon：可执行与性能的 分界	21
5.4	一个数值反例：FMA 与乘加	11	11	限制、可复查性与结论	22
5.5	完整 Program 的阶段与绑定	11	11.1	解释结果时的四项限制	22
5.6	精确目标文档	12	11.2	下一步的可检验目标	23
5.7	表示变更的验收义务	12	A	术语、源码入口与完整示例	24
6	编译器：从局部化检查到目标源码	12	B	结果来源与历史身份	25
6.1	分层拒绝与反馈	12	C	复现边界与报告构建	25
6.2	源码生成与下游工具链	13			
6.3	Target 与生成路线如何保持分离	13			
6.4	从 Finding 到物理资源观察	13			
6.5	Corpus Gate 的含义	14			

1 问题、研究对象与论点

以按行计算的 Softmax 为例，公式确定了每行应得到什么结果，却没有规定一个 GPU 程序处理多少行、由哪些线程完成归约，也没有规定中间值放在哪里。智能体可以尝试多种安排；其中有的地址或同步关系不成立，有的虽然算对，却比原实现慢。要继续改进，智能体需要知道具体是哪项选择出了问题，而不只是收到“编译失败”或一个延迟数字。

本报告研究的是这样一套编写环境：智能体修改明确的执行计划，编译器在生成代码前检查可判断的关系，设备评测再回答结果是否正确、性能是否改善。当任务反复暴露出表达或分析缺口时，维护者改进编译器，并在后续任务中检验这项改动。贯穿全文的问题是：

核心问题 怎样把优化发现变成带条件的可执行机制，让 Agent 在目标硬件上重新实例化，并在相同语义、参考权限和声明预算下减少搜索成本、保持性能竞争力？

CAKE 论文提出了编译器与智能体协同改进内核的研究路线 [1]。open-cake-ir 沿着这一问题独立实现了程序表示、编译检查和实验流程，并把不同硬件上的观察保留在各自的任务与测量范围内 [18]。

1.1 背景：公式、执行计划与智能体反馈

数学公式与设备程序之间隔着一层执行安排。以内核中的行归约为例，行块大小改变了每个程序实例承担的工作；线程分工影响归约时的通信；中间值的存放方式又影响寄存器与共享内存压力。编译器可以检查一部分地址、数据流和同步关系，物理资源分配与速度则要到编译产物和设备上观察。因此，一个候选的反馈需要指出检查到了哪一层，以及问题落在哪个操作或资源上。

这些观察还决定改动应当留在哪个位置。某种分块在当前形状上更快，可以先保留为任务候选；换到另一目标时，需要重新选参数并验证；若同类改写在不同任务中反复有效，才有理由提炼带适用条件的编译器变换。报告将依次展示这三层工作所依据的程序、诊断和设备记录。

1.2 变化中的工具链与编译能力

固定 GPU 目标并不固定软件编译环境。NVIDIA 的 CUDA Toolkit 12.1 与 12.2 分别列出不同的 nvcc 组件版本，发布说明也记录了编译器诊断与 PTX 接口的变化 [2, 3]。因此，算子候选的编译和评测需要连同工具链版本一起固定；硬件目标名本身不足以标识一次实现条件。

智能体可以连续提出新的分块、存储与同步选择。多个任务若反复遇到同一类表达、合法性或后端生成缺口，修正单个候选无法消除共同的编译器限制。研究者或维护 Agent 依据留存的计划、诊断和设备观察确定责任层，再把修复、反例及相应分析一同送过语料门与审查。后继任务在新编译器版本上验收这项能力。

1.3 CAKE 论文提供的研究起点

CAKE 让智能体编写一份带类型的硬件执行计划。计划写明资源、线程角色、数据访问和同步；屏障地址与流水线阶段位等机械细节由编译器推导。编译器先检查程序安全、硬件约束、数据一致性和调度语义，再把有希望的候选送去编译与设备评测。任务中反复出现的失败，可以推动后续增加原语、改进检查规则或校准性能估计。CAKE 的重点因此不只是生成更快的内核，也是让编译环境随着任务一起改进 [1]。

论文在 B200 上用固定形状的 Flash-KMeans 任务比较了两种编写环境：一组由智能体编写 Cake

IR，另一组直接编写 CUDA/PTX；目标的低层实现都不向作者开放。两组各运行三次，共同使用 80M provider-token 截点。这个实验比较的是整套编写环境，包括表示、检查和反馈，并未单独测量 IR 语法的作用。论文中的 KDA 服务及其他性能数字也属于原研究的结果；本文讨论本仓库时，会另列其独立运行的证据。CAKE 论文覆盖的目标是 NVIDIA Ampere 至 Blackwell，非 NVIDIA 平台的实现和测量需要另建证据 [1, 18]。

1.4 本工作的定位与证据范围

Compiler 拥有完整 Program、叶子 Schedule、合法性检查、确定性改写与 lowering；Lab 负责材料、作者与研究处理。工作围绕可复用机制及目标适配组织，具体问题与分支分工见第2节。原 CAKE 的 IR、诊断和协同演进设计是明确承继的基础。

历史实现章节提供程序表示、检查流程和有界设备案例。研究增量则需要单独检验：带条件改写是否减少关联修改错误，来源信息是否帮助目标重选，以及这些收益能否覆盖发现、工具实现与维护成本。新增设计不将工程 Run 重标为受控消融，也不把接受更多候选等同于更强的正确性保证。

2 共同研究主题与跨平台实验

2.1 从优化发现到条件化机制

本项目以 Compiler 为核心，研究优化发现怎样成为后续任务可用的机制。一次发现同时包含可复用的数据流改写和依赖目标的决策：例如分块可以限制活动数据与资源压力，但 tile、执行组和流水线需要结合目标重新选择。机制保留前后程序、语义条件、目标实现条件、可调决策、反例和版本范围；收益假设仍由实机确认。[共同研究约定](#) 统一各硬件分支的问题与对照，具体实验仍由 Workload、Run 和 Study 的现有职责承接。

研究区分三项假设：表示与工具是否改善有限预算搜索；来源经验和可执行化是否各有增量；经审查的系统后继是否帮助未参与修复的任务。多目标接入、单次本机加速与一个可调用 pass 分别提供局部证据，不能直接替代这些受控比较。

2.2 相关路线与增量贡献

CAKE 和 TIRx Harness 已研究 Agent 编写接口、编译分析与优化反馈 [1, 4]；Exo 2 与 MLIR Transform 提供可组合、可扩展的调度或编译变换 [5, 6]。AutoTVM 与 MetaSchedule 研究调优、知识迁移和可扩展搜索空间 [7, 8]。AKG kernel Agent 分离优化 sketch 与具体 DSL 生成 [9]；KernelBlaster 与 AccelOpt 把经验形成和后续利用作为研究对象 [10, 11]。本项目将这些能力明确作为基础和对照，贡献需要落到具体机制、目标适配方法及可重复的效果上。

AI lowering 增加了一条重要比较路线：从既有 Triton 程序和固定执行合同生成 PTX，让模型承担更多底层实现决策 [12]。该论文的原 Volta 限定候选与数值验收候选有不同的最佳结果，提示验证覆盖可能限制搜索；最佳候选表不构成相应程序集合的理论性能上界。原 Volta 的模型内保证与后续扩展的保证需要分别解释 [13, 12]。符号检查、有限设备测试、计时质量和实际加速也各有自己的验收范围。

表 1 三个作者接口的比较边界。路线只有通过精确目标的工具链、加载与评测准入后才进入实机对照。

比较	固定与变化	解释范围
Cake 与原生 DSL	同源材料、语义等效工具和同一下游工具链；公共 oracle 与预算一致	表示和诊断组合的增量；共用内部引擎时注明作者接口效应
DSL 与直接低层	明确常量、grid、线程数及 ABI 的可变范围；低层路线单独取得资格	lowering 和指令选择自由度，不能全部归因于 IR
非 LLM 与 Agent	同动作集合、参数域和设备评测预算	选择策略的增量；自由编写代码另作系统评估

共同能力子集用于隔离因素，完整合理能力用于评估覆盖与强实现差距。给定优化源码与只给数学规格具有不同参考信息。模型、权限、反馈、测量和主要预算在研究前固定；不显著的差异不自动证明等效。原生接口或表达形式也不决定理论上更高的性能，研究重点是有限预算内实际找到并独立确认的结果。

2.3 独立原生发现与后继晋升

IR 保持默认开发接口。具体假设暴露表达或 lowering 缺口时，Lab 可以在已经准入的路线启动独立原生工程 Run，保留同一 Workload、精确 Target、oracle 和测量要求，并声明预算、继承材料和参考访问。冻结 Run 内不切换路线，也不向原研究回填原生成绩。

发现按责任层处理：何时使用和如何选参进入 Lab；可复用改写进入 Compiler pass；兑现缺口进入 lowering；可建模的非法行为进入 Verifier；真实表达缺口才共同扩展 primitive、类型、效应和分析。审查与验证完成后，后继提交接受新任务检验。No promotion 是有效结论。新 Schedule 或 pass 必须重新验证，不能继承来源低层实现的正确性或速度。人工提炼、自动提炼与使用工具是不同过程，分别记录发现、实现、审查和维护成本。

**图 1** 发现、晋升与目标复用的研究流程。各阶段有独立证据，图中没有实测收益。

2.4 各硬件分支回答同一主问题

各分支选择自身已能可靠评测的任务，贡献不同技术差异的证据。NVIDIA 优先提供结构性发现、强原生对照和显式存储/同步案例；MetaX 检验联动分块、产物资源与测量边界；Hygon 检验目标侧重选、数值行为及寄存器和 scratch；AMD 验证各精确目标的执行宽度、资源与工具链差异；Metal 检验 threadgroup 分工、存储及表达/评测覆盖。任何平台都可成为来源或目标，main 维护共享语义和研究口径，不替平台宣布设备资格。

新实验在已有计划和 Run/Study 输入中说明假设、机制与任务、对照、参考权限、预算、测量合同和停止/确认规则。新优化 Run 默认显式绑定三小时总预算，确认资源包含其中；专项准入探针另设有界预算。研究若使用不同预算，在执行前对各组一致声明；既有 Run 不追改。这是实验设计约定，不增加一种 Study 或执行器。

发现、目标适配与测试分开。参与过机制或参数选择的任务属于开发证据。未见形状、未见算子族和新目标分别报告；重复 Run 和计时样本不充当新增任务。按卡位和时间分块，保留失败与未达到终点的结果。墙钟、token、CPU 准备、编译、排队和设备用量分别记账。强种子通过验收

后，再检验完整 Program、模型或服务的收益。

3 设计原则与实现边界

CAKE 在附录 B.1 给出八项 IR 设计原则[1]。本仓库沿用这些原则来约束新的执行计划语法、检查规则和后端能力。表2列出它们在实现中的具体落点。

表 2 P1–P8 对本仓库的实际约束。具体语法和目标准入仍以源码合同为准。

原则	本仓库的实现落点
P1 易于编写	Python 前端提供变量、表达式和角色作用域；网格细节由计划和编译器推导。
P2 性能透明	影响性能的分块、执行组、存储、访问和同步选择保留在 Schedule 中。
P3 规范形式	Python 与 JSON 进入同一套类型和操作模型。
P4 静态类型	类型、操作数与形状错误在构造和检查阶段定位。
P5 可分析	IR 显式保留 Verifier 与资源分析需要的数据和执行关系。
P6 语料门	新语义同时接受既有正例与反例的 Corpus Gate 检查。
P7 分析一致	数据模型改动同时更新对应的合法性与资源分析。
P8 硬件根据	目标指令、存储和同步承诺由 Target 与后端说明其适用范围。

硬件事实由精确目标文档声明。遇到未建模的资源上限，系统明确报告覆盖缺口；遇到目标不支持的指令，后端给出拒绝原因。这样，一份计划不会因为名称相近而悄悄降到另一架构。静态检查负责在已建模的范围内提前发现问题，真实输出与速度仍由设备评测确认。

编译器还要保持独立。它解释程序和目标，却不读取研究分组、作者工具或证据库。一次运行固定编译器与任务，运行中比较候选；编译器自身的修改在后继提交通过语料检查后，才用于下一轮任务[14, 17]。

4 系统合同与职责边界

4.1 从任务到可审计结论

一次内核优化从任务合同（Workload Contract）开始。它规定输入、输出、数学定义和数值容差，给所有候选提供同一个判题标准。智能体提交的执行计划（Schedule）则写明这个内核怎样分工、读取、计算和同步；需要多个内核时，完整程序（Program）再说明阶段顺序及张量如何交接。

运行规格（RunSpecification）把这次尝试使用的源码、目标硬件、可见材料和预算固定下来。如果目的是比较两种编写条件，研究计划（StudyPlan）还要在运行前分配组别、终点和分析规则。表3列出这些事实各由谁负责；有了这样的分工，读者才能从一次候选追溯到它实际参加了哪项任务与评测[14, 28]。

图2展示了典型运行路径。实验系统调用编译器检查候选，再把可运行产物交给评测；评测与证据记录一起返回下一轮。编译器也可以脱离实验系统单独使用，例如在没有 GPU 的机器上检查 FMA 示例。模块依赖与源码入口见 [架构章节](#)和[职责地图](#)。

表 3 从任务到结论的负责位置。每一行给下一阶段提供事实或产物。

对象或模块	拥有的事实	对外给出的产物
Workload	数学、形状、输入分布、外部答案	不随候选改变的验收合同
Compiler	IR 类型、Target、合法性、后端准入与生成	Assessment、Finding、目标源码
Lab	材料、候选搜索、预算与 Run 控制	已提交候选和过程事件
Evaluation	外部正确性、目标计时质量和剖析	独立的检查、计时和 profiler 收据
Evidence	原始文件、事件、回放与审计	可由原始材料重建的报告
Study	处理分配、估计量和缺失策略	限定研究问题的分析结果

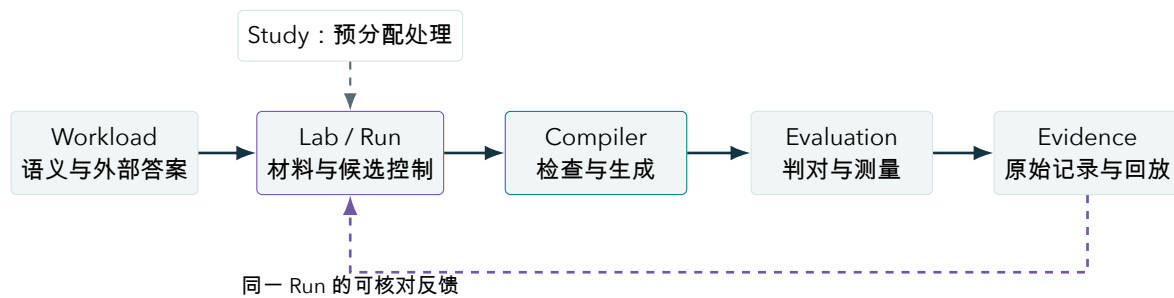


图 2 一次运行中的候选路径。任务提供外部答案，实验系统管理候选，编译器生成源码，评测和证据模块把可核对的结果送回智能体；上方的研究计划只在需要分组比较时使用。

4.2 编译器：检查程序，再生成目标源码

编译器读入完整程序及其各阶段计划，在一个精确目标上检查类型、张量绑定、数据依赖和硬件约束。检查结果把两个问题分开回答：`Assessment.accepted` 说明计划是否符合通用语义规则；`lowering_eligible` 说明所选后端能否把它生成源码。比如，计划本身可以表达一种操作，某个后端却尚未实现对应指令，此时反馈会指出后端能力缺口。

通过检查的计划由后端确定性生成可阅读的源码。编译器中的显式改写也必须给出完整的新候选，或说明为何拒绝改写。这些判断只依赖程序、目标与编译器版本；实验组别和设备测量由后面的模块处理 [14, 15]。

4.3 实验系统：固定一次优化的条件

一次运行开始前，实验系统先核对任务、编译器、执行环境、基线和预算，再向编写内核候选的智能体提供获准查看的任务材料。搜索中，智能体提交候选，控制器记录剩余的编译与评测次数。搜索结束时，系统从已有候选中固定一个提名，用独立的新样本确认其正确性与性能；确认结果出现后，提名不再更换。

普通工程运行只需自己的运行规格。研究比较则由研究计划提前分配处理组，并让各组沿用同一执行与评测流程。旧实验使用的 CampaignLock 保留为输入适配层，历史数据仍按产生它的版本解释。这使智能体能看什么、能调用什么工具、何时停止，都在结果出现前确定。

4.4 评测与证据：先判对，再解释速度

评测模块先用任务合同生成输入，与外部参考逐项核对输出。候选算对之后，才按目标设备的计时与状态重置约定采集样本，并在需要时运行性能剖析。搜索读数、独立确认和剖析结果各有单独收

据。若计时波动超过质量门，记录说明的是测量尚不足以判定快慢；候选的数值正确性仍按先前的检查结果保留。

证据模块保存候选文件、事件顺序、评测收据和终态。独立审计可以在不再调用智能体服务或 GPU 的情况下重建运行过程；研究分析只使用通过审计的结果。对多阶段程序，评测还要核对每次启动的顺序与最终输出，并按实际程序区间计时。单内核的计时收据只能回答那次内核调用的问题[17]。

4.5 两个时钟：候选搜索与编译器演进

候选和编译器沿两条时间线变化。在一次运行里，任务、目标、编译器及测量规则保持固定；智能体改变的是执行计划。检查与设备评测指出下一份候选该改哪里，也让不同候选的结果可以在同一条件下比较。

如果多次运行都遇到相同的表达或检查缺口，维护者才在运行之外修改编译器。新版本先经过语料测试与代码审查，再用于下一轮任务。这样，旧运行的结果仍对应旧编译器，新运行才能检验编译器改动有没有解决原来的问题[14, 17]。这一外环的迭代速度可以按缺口被定位到后继能力通过验收所需的时间衡量，同时记录修复在未见任务上的复用和回归情况；当前仓库尚无与人工维护流程匹配的迭代耗时对照。

5 程序表示：任务、完整程序与执行计划

5.1 显式的计划，而非新的布局代数

对每一行做 Softmax，任务规定的是结果：若 $m = \max_k x_k$ ，则 $y_j = \exp(x_j - m) / \sum_k \exp(x_k - m)$ 。执行计划要补上公式没有写出的事：每个程序实例处理几行，哪些执行组负责归约，中间结果放在哪里，以及按什么坐标写回。图 3 以仓库示例中的 64 行块展示这些选择。

Schedule 把操作、缓冲区、执行组、访问坐标、循环和同步关系存成带类型的结构。编译器因此可以检查：读取和写回是否指向同一行，两组是否争写同一输出，目标是否接纳所需指令。它要求作者写明具体的存储与访问承诺，却没有另设一套通用布局代数；Triton 等下游工具链继续负责物理布局、寄存器分配和最终指令选择[15, 14]。

当计算需要多个内核时，Program 将各阶段接成一个完整程序，声明它们共享的输入输出、执行顺序和张量绑定。此时验收对象是程序的最终输出与完整调用区间，而非某一个阶段的单独速度。

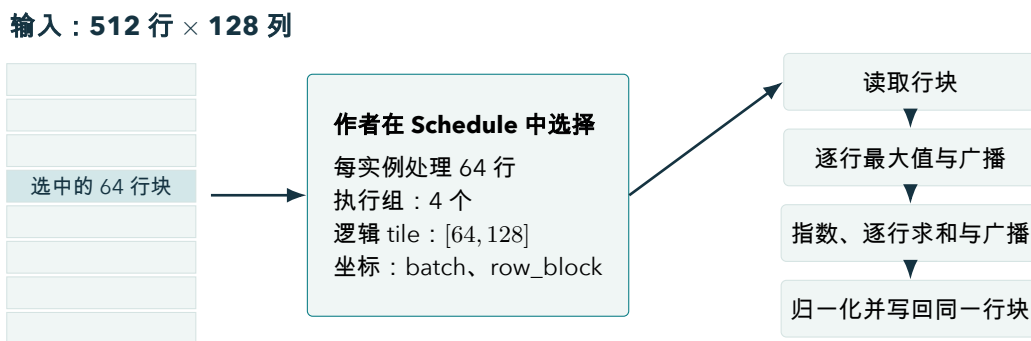


图 3 同一 Softmax 公式对应的一份具体执行计划：每个实例处理 64 行，由四个执行组完成读取、归约和写回。图改绘自仓库原图，展示的是候选结构，而非计时结果。

5.2 受限 Python 前端与同一规范 IR

示例把 Python 前端导入为 `cake`，用它声明 `@cake.schedule` 和张量参数；函数的第一个形参 `lm` 是 Schedule 上下文的语法名字：前端识别它的方法调用，不会执行一个运行时的 `lm` 对象。仓库没有为 `lm` 规定缩写含义；把形参及函数体中的引用一起改名为 `ctx`，仍会得到同一份规范 Schedule。`lm.program` 选择行块，`lm.role` 指定执行组，角色作用域内的调用描述读取、归约、广播和写回。前端解析受支持的 Python 语法树，并为诊断保留源码位置；它不会运行源文件的导入语句或函数体[15]。下面摘录的是 [当前 Softmax 示例](#) 中的实际操作：

```
compute = lm.role(execution_groups=[0, 1, 2, 3])
row_block = lm.program(x, axis=0, dimension=1, tile=64)
batch = lm.program(x, axis=1, dimension=0, tile=1)
with compute:
    x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
    rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
    shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
    weights = lm.exp(shifted, id="exponentiate")
    rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
    y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
    lm.store(y[batch, row_block, :], y_tile, id="store_y")
```

示例输入输出都是 [8, 512, 128] 的 FP32 张量，目标是 `sm_100a`，生成路线为 Triton。`tile=64` 使每个实例负责 64 行；四个执行组完成同一份行块上的工作。行最大值和行和先广播回列维度，再参与逐元素计算；最后仍按原来的行块坐标写回。换一种分块或分工，编译器会重新检查计划，设备评测再判断它是否算对、是否更快。

作者在 Python 装饰器中无需手填任务内容的哈希。独立调用编译器时，可以直接检查计划；进入实验系统后，系统先核对目标、生成路线和公开张量接口，再绑定这次运行冻结的任务。这样，任务身份由实验输入决定，作者只编辑实现方案。具体准入规则见 [前端合同](#)。

5.3 从 Python Schedule 到 Triton：一次实际生成

对上面的 Softmax，固定源码 677eb444 的 Compiler 先把 Python 语法树转换为带类型的 Schedule，再检查形状、访问、操作依赖和目标约束。本例的结构检查通过，Triton 生成路线可用；报告另给出 `REGISTER_PRESSURE` 与 `RESIDENCY_BOUND` 两条提示。随后执行 `lower`，得到下面的 Triton 内核体节选；两处较长的地址表达式只按页宽重新折行。代码中的 `CAKE_OP` 注释保留了原计划的操作 ID，便于从生成代码找回 `load_x`、`row_max` 等编写位置。

对照速览：`lm.program` 的 64 行块变成程序 ID 与行偏移；`lm.load` 的访问变成地址、边界掩码和 `tl.load`；行归约与广播变成 `tl.max/tl.sum` 和 `[:, None]`。编译器还把四个执行组写入启动参数 `num_warps=4`。

```

row_block = tl.program_id(0)
batch = tl.program_id(1)
row_block_offsets = row_block * BLOCK_ROW_BLOCK + tl.arange(0, BLOCK_ROW_BLOCK)
x_d2_offsets = tl.arange(0, D_X_2)
y_d2_offsets = tl.arange(0, D_Y_2)

# CAKE_OP:load_x
load_x_ptrs = (
    x + batch * N_ROW_BLOCK * D_X_2
    + row_block_offsets[:, None] * D_X_2
    + x_d2_offsets[None, :]
)
x_tile = tl.load(
    load_x_ptrs,
    mask=row_block_offsets[:, None] < N_ROW_BLOCK,
    other=0.0,
    eviction_policy="evict_last",
)
# CAKE_OP:row_max
rowmax = tl.max(x_tile.to(tl.float32), axis=1)
# CAKE_OP:shift
shifted = x_tile - rowmax[:, None]

# CAKE_OP:exponentiate
weights = tl.exp(shifted)

# CAKE_OP:row_sum
rowsum = tl.sum(weights.to(tl.float32), axis=1)
# CAKE_OP:normalize
y_tile = weights / rowsum[:, None]

# CAKE_OP:store_y
tl.store(
    y + batch * D_Y_1 * D_Y_2
    + row_block_offsets[:, None] * D_Y_2
    + y_d2_offsets[None, :],
    y_tile,
    mask=row_block_offsets[:, None] < D_Y_1,
)

```

摘录省略了函数签名和宿主启动包装。完整文件还核对输入的形状、dtype、设备和连续性，随后将 $512/64 = 8$ 个行块与 8 个 batch 组成 (8, 8, 1) 网格，传入 BLOCK_ROW_BLOCK=64、num_warps=4 和 maxnreg=96。其中的 tl 是实际导入的 triton.language；前面的 lm 只是 Cake 前端识别的形参名。

读者可在固定源码的根目录运行下列命令，查看包含函数签名、输入检查和启动包装的完整生成文件。这个步骤只生成 Python/Triton 源码，设备编译、正确性和速度另由评测确认。

```

PYTHONPATH=src python3 -m open_cake_ir.cli compiler assess \
    examples/python/softmax.py --format text
PYTHONPATH=src python3 -m open_cake_ir.cli compiler lower \
    examples/python/softmax.py --output /tmp/cake-softmax-generated.py --format text

```

5.4 一个数值反例：FMA 与乘加

FMA 说明为什么公式相同，还要写清计算语义。 $a \times b + c$ 可以先乘后加、舍入两次；融合乘加只在最终结果处舍入一次，某些输入下两者会得到不同的 FP32 值。当前 FMA 示例处理 $[8, 128]$ 输入，显式请求 `ptx.fma.rn.f32`。图4把任务公式和这份计划的分工放在一起：公式规定应算什么，计划规定八行怎样分配、数据怎样读取和写回 [15]。

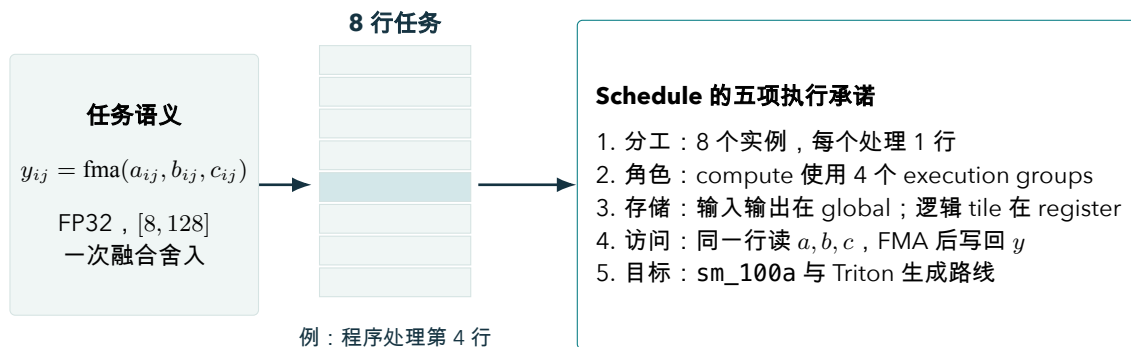


图 4 FMA 示例的任务与计划。图按仓库原图重绘；高亮行代表一个程序实例要处理的数据，逻辑寄存器 tile 的长度并非每线程物理寄存器数。

5.5 完整 Program 的阶段与绑定

QSA Program v6 是一份可检查的多阶段实例。其任务在 $B = 1, T = 32768$ 时读入 q, k, v 、索引查询与索引键，最终写出 BF16 的 output，形状为 $[1, 32768, 32, 128]$ 。Program 中的五个阶段各有一份独立 Schedule；下表直接按合同中的张量绑定与阶段顺序展开，而不是只用“池化、选择、注意力”概括整个程序。

表 4 QSA Program v6 的实际五阶段数据流。形状来自固定 $B = 1, T = 32768$ 的任务与 Program 合同；本表描述程序结构，不代表第8节历史候选的计时。

阶段	读取的张量	写出的张量及作用
pool	index_k	pooled: FP32 [8192, 128]，每 4 个索引键池化一次
layernorm	pooled, k_norm_weight	normalized_keys: FP32 [8192, 128]
score_topk	index_q, normalized_keys	block_indices: INT32 [32768, 512]，每个查询选 512 个块
expand	block_indices	token_indices: INT32 [32768, 2048]，展开为 token 位置
attention	q, k, v , token_indices	output: BF16 [1, 32768, 32, 128]，只读取选中位置

例如，合同中的 expand 节点把 score_topk 产生的块索引接到自己的 Schedule，再写出供 attention 消费的 token 索引。下面保留原 JSON 的数据流字段，省略身份校验字段：

```
{
  "id": "expand",
  "schedule": "corpus/schedules/qa-expand-t32768.json",
  "inputs": ["block_indices"],
  "outputs": ["token_indices"],
  "views": {"block_indices": "identity", "token_indices": "identity"},
  "depends_on": ["score_topk"]
}
```

编译器逐阶段检查 Schedule，也检查 Program 的公共输入输出、中间张量形状与 dtype、生产者和依赖关系；例如 `block_indices` 必须先由 `score_topk` 产生，`expand` 才能读取。合同还声明了输入的 singleton 轴视图，使阶段局部 Buffer 与公开张量对应。五个阶段分别生成目标源码，评测核对最终 output，计时覆盖完整调用序列。此处的 v4 是固定源码中的编译验证实例；第8节的 127.8 ms 历史 QSA 结果绑定它自己的旧版程序与设备收据。

5.6 精确目标文档

同一份执行计划语言要面对不同硬件，具体能力由目标文档（Target）说明。当前八个目标分属 NVIDIA、Apple、AMD、Hygon 和 MetaX；每份文档分别声明线程组宽度、存储空间、可用指令、代码对象与资源限制。编译器按目标的精确 ID 准入，缺少的硬件事实报告为“未建模”。Triton、Metal、原生 CUDA 与 CuTe DSL 则是生成源码的路线；其中 Triton 服务多个厂商，不以厂商名划分公共 IR[16, 14]。

5.7 表示变更的验收义务

新原语会改变作者能写什么，也会改变编译器必须检查什么。因此，一项 IR 扩展要同时说明结果类型、读写效果、合法性规则、分析所需信息和目标上的实际行为，再用正反用例检查这些关系。对于能够由现有原语组合的算子，先建立任务合同和参考实现；遇到确实无法执行的执行行为，再扩展语言与后端[1, 15]。

6 编译器：从局部化检查到目标源码

6.1 分层拒绝与反馈

编译器先把作者的程序构造成带类型的 IR，再检查程序中的关系是否成立。例如 FMA 需要三个输入；若候选漏掉第三个读取，检查结果会在 `operations[3].reads` 指出 `ELEMENTWISE_ARITY`。作者据此补上缺失的数据边，而不必先编译、运行 GPU 再从错误输出猜原因。其余检查归入四类：

`schedule_semantics` 调度语义

`hardware_conformance` 硬件符合性

`data_consistency` 数据一致性

`program_safety` 程序安全

一条诊断（Finding）带有稳定代码、位置、类别及阻断范围，Python 前端还能把位置指回源文件。建议性提示另存，不改变候选的检查结论。通用规则通过后，后端还会检查这条生成路线能否支持候选所用的操作与硬件特性。通过这些门，意味着候选可以进入下一步编译；设备上的答案与速度仍要分别测量[14, 17]。

6.2 源码生成与下游工具链

检查通过后，后端生成一份可阅读的目标源码。选择 Triton 时，这份源码还要交给对应厂商的 Triton 工具链；Metal、原生 CUDA 和 CuTe DSL 各有自己的生成与编译入口。目标文档声明最终代码对象的类别，执行器按该类别加载产物。这样，从执行计划到 GPU 结果的过程可以逐段追踪：源码是否生成、工具链是否编译、设备是否可加载、输出是否算对、计时是否可信，各有自己的记录 [14]。

6.3 Target 与生成路线如何保持分离

同一代码生成机制可以服务不同厂商：当前 Triton 路线就接入了 NVIDIA、AMD、Hygon 和 MetaX。厂商身份由精确目标文档声明，后端只说明自己能生成哪类代码对象。表5列出这些关系。新增目标时，先核对它声明的指令、资源和代码对象，再验证所选后端与工具链；添加一个目标名并不会自动带来设备执行资格。

表5 固定源码中的目标与生成路线。代码对象来自各自 Target 文档；设备证据另见第8节。

Vendor	精确目标 ID	代码对象	当前主要源码路线
NVIDIA	sm_100a, sm_103a	CUBIN	Triton; 原生 CUDA; CuTe DSL
Apple	apple_gpu_family7, apple_gpu_family8, apple_gpu_family9	Metal archive	Metal
AMD	gfx1151	HSACO	Triton / ROCm
Hygon	gfx938	HSACO	Triton / DTK-HCU
MetaX	xcore1002	MCFATBIN	Triton / MACA

目标文档还拥有线程束或波前宽度、资源上限和指令事实。某项上限尚未声明时，分析会把它列为未建模，而不是借用另一型号の数値。MetaX 提供的 `triton_arch=80` 只是适配工具链 API 所需的值；物理目标仍是 `xcore1002`。这种区分使报告和代码生成都能说清实际使用了哪台设备 [16, 14]。

6.4 从 Finding 到物理资源观察

计划通过静态检查以后，作者还需要了解它在机器上付出了什么代价。Schedule 中声明的逻辑寄存器 tile，并不是每线程实际分到的物理寄存器；后端还可能增加暂存、栈与共享内存。工具链编译后的资源报告可在不运行 GPU 的情况下收集。若候选正确但偏慢，目标设备上的性能剖析进一步帮助作者提出关于访存、占用率或工作分配的假设。

这三种反馈各回答一个问题：计划关系是否成立，编译产物用了多少资源，设备执行时发生了什么。它们绑定同一候选的源码和产物身份，方便作者沿着观察逐步修改。分析缺少目标校准时，报告保留未知状态。细节见 [编译资源分析与校准边界](#)。

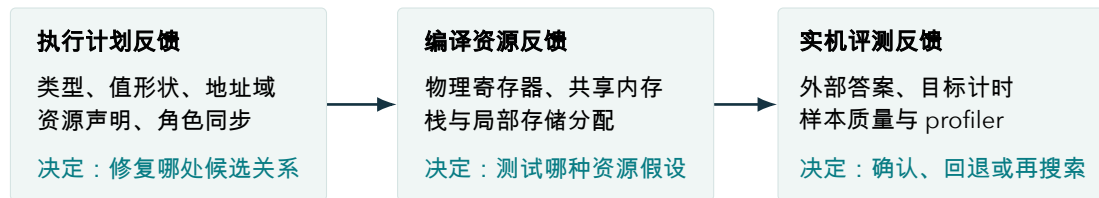


图 5 作者沿着计划、编译产物和设备运行三个层次读取反馈。图按 [仓库既有说明图](#) 改绘；每一层都为下一次修改提供不同的观察。

以下命令可在项目根目录离线检查 FMA 示例，并生成 Triton 源码。它让读者先观察 Assessment 与生成结果；工具链编译、GPU 判对和计时属于后续步骤：

```
PYTHONPATH=src python3 -m open_cake_ir.cli compiler assess \
  examples/python/fma.py --format text
PYTHONPATH=src python3 -m open_cake_ir.cli compiler lower \
  examples/python/fma.py --output /tmp/cake-fma.py --format text
```

6.5 Corpus Gate 的含义

当前固定源码 677eb444 的语料门有 **179/179** 项符合预期。它逐项对照已审查的接受或拒绝结果，帮助维护者发现编译器语义变化带来的回归。覆盖报告同时指出两个已声明但尚无该门检查的目标：Apple family7 和 family9。通过数因此只描述这组软件用例；设备运行与性能由各目标自己的实验继续验证 [19, 17]。

7 实验方法与证据等级

7.1 一次 Run 的四阶段

一次运行从固定任务开始，沿四个阶段推进：

1. 作者提出结构不同的执行计划或完整程序，记录每次改了什么；
2. 编译器先检查类型、合法性和后端支持，再用有目标校准的成本估计安排设备评测顺序；
3. 对可运行候选核对完整输出；正确后按目标设备的协议计时，并在需要时采集剖析数据；
4. 根据具体诊断继续修改候选，或将反复出现的编译器缺口留给后继版本处理。

成本估计节省搜索时的设备时间，最终接受仍取决于外部答案和实机测量。每种目标硬件分别声明计时器、测量区间与样本前的设备状态；缺少这些条件时，报告保留计时覆盖缺口。

7.2 双时间尺度与 Finding 的负责位置

图6展示了两种改动的节奏。一次运行只改变候选程序，任务、编译器、目标和测量规则保持原样。运行留下的错误、正确性收据与性能样本，为下一轮候选提供依据。运行结束后，维护者再查看反例：问题是候选写法、检查规则、生成路线，还是缺少一种表达？需要修改编译器时，改动先成为新提交，通过语料门和审查，再进入下一次运行。有些性能差距只适合继续探索候选，记录“暂不提炼通用变换”也是结论。

一份 C550 M17 的实际运行展示了这条路径。在固定源码 896e0887 下，作者进行了十个回合，九份候选被拒绝。第二轮的 ACCESS_TILE_MISMATCH 直接指向不一致的访问映射；第六轮

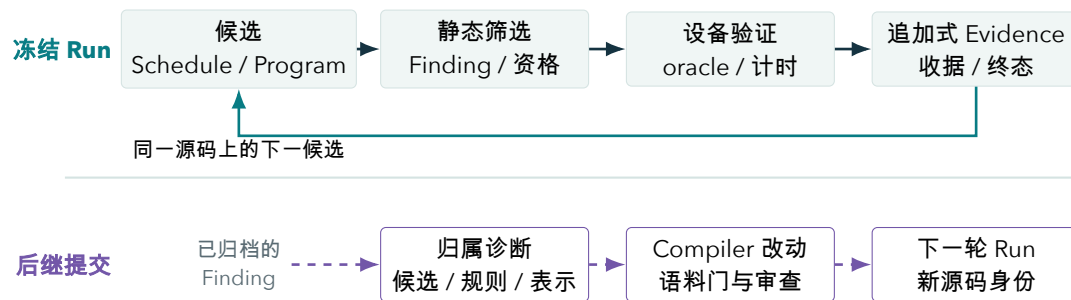


图 6 候选在固定编译器的一次运行中迭代。下方的虚线表示运行结束后，已归档的问题可能推动编译器改动；新版本通过审查后才用于下一轮。

产生的候选后来通过五类输入的独立正确性检查和 MCPTI 成对计时。固定基线与候选的中位数分别为 12.032 和 11.008 μs 。这份记录覆盖了反馈、筛选、确认到终态的过程；计时只对应那次 `local_serialized` 容器环境。详情见[架构案例](#)。

7.3 研究对照的单位和参考访问

研究比较以一次独立运行为单位，而不是把同一运行中的多个候选当作重复样本。干净起点允许作者看任务规格、外部答案和高层代码，目标的完整低层实现则留在作者视野之外；研究已知内核的复现时，参考实现可以成为允许的材料。比较 Cake 和直接 CUDA 编写环境，还须固定模型、任务、预算与评测规则。搜索读数用于挑选候选，结论取自候选固定后的独立确认。基础设施中断在报告中记为缺失，保留它已经消耗的成本 [18, 28]。

7.4 证据保存、回放与结论资格

每份候选、评测收据和终态都按事件顺序追加保存。审计者可以离线重算候选选择与停止状态，并检查报告是否来自这些原始材料。还要分别核对归档内容能否完整回放，以及当时的写入与发布过程是否满足文件监管要求。前者说明记录可读，后者决定它能否用于晋升或研究估计。协议将两项分别记为 `archive_integrity` 与 `filesystem_custody_verified`。历史运行始终按当时的源码与协议审计 [17]。

7.5 结论阶梯

一份结果能支持多强的说法，取决于它实际通过了哪些步骤。表6 列出从计划检查到研究比较所需的证据，也供后面的平台案例对照。

源码版本由干净的 Git 提交确定：编译器身份写作 `open-cake-ir@<commit>`，执行器还要加上精确目标。本文固定的提交用于说明当前软件；后面的历史测量仍保留各自运行时的提交、样本与监管记录。

8 实现覆盖与代表性设备案例

8.1 平台覆盖是分层事实

仓库已接入五家厂商，但“能生成代码”“设备算对”和“计时可比较”是不同阶段。表7先给出每条路线已经到达的范围；具体数值在各自的任务、设备和基线之内解释。各平台的原始报告与收据仍是结果来源 [14]。

表 6 从计划检查到研究比较，各种结论需要的证据。

可陈述的结果	必要证据	尚需另行验收
结构与目标准入	typed IR、精确 Target、Finding 与后端预检	工具链编译和加载
设备正确性	完整输出对外部 oracle，覆盖规定形状与分布	计时质量和性能收益
局部性能	固定候选与基线、共同计时区间、质量通过	跨任务或跨硬件泛化
Program 或服务收益	完整调用路径、dispatch 与应用层验收	Agent 处理的因果效果
研究归因	预注册处理、独立 Run、共同确认与缺失规则	未测试任务与平台的外推

表 7 截至报告快照的代表性平台能力与主要解释边界。

平台	已报告的执行范围	需保留的边界
NVIDIA	B300 的单 Kernel、部分完整 Program；B200 另有固定案例	FlashInfer 外部比较有通过与未通过质量门的边，按任务解读
Apple	Metal 固定算子与 TaskLab 优化记录	已验收设备和计时口径不推广到全部 family 或完整 Program
AMD	gfx1151 设备调查与 smoke	计时器绝对时间未对齐，尚无合格的完整 Agent 收益结论
Hygon	BW1101 / gfx938 固定任务与 HIP 路线	短 Kernel 的计时分辨能力限制部分比较
MetaX	C550 固定单 Kernel 及若干完整输出；单 Kernel 配对计时和 profiler	完整 Program 普通 Run 计时与完整 provider 闭环仍需验收

8.2 NVIDIA：任务族覆盖与外部比较

在 B300 上，FlashInfer 逐任务综述 为每个固定形状保留原起点、改写产物和外部实现。截至 2026-09-21，26 个任务中有 16 个完成三方数值比较。按综述选定的代表产物，对外部实现的计时结果分为：2 项合格领先、6 项未检出达到预设门槛的差异、4 项合格落后、4 项缺少合格的外部计时；其余 10 项还在完成外部性能比较。这个分布展示任务进度，不能合成为一个平台加速比。

逐任务记录更能说明优化发生了什么。任务 022 的四组候选对外部实现取得合格领先，但它与原起点之间的计时质量不足，原起点仍保留为比较对象。任务 003 的整行改写相对中间候选改善，面对外部实现仍落后。这些结果把“相对自己的上一版进步”和“达到外部实现水平”分成两件事。独立 NCU 剖析可帮助解释工作分配与资源使用；计时判断仍依据原来的 CUPTI 样本及质量门 [20]。

B300 的服务实验、CTA 宽度验证、FlashInfer 改写和 CAKE 任务复现使用不同基线与协议，分别放在报告中。B200 的 FMA、状态操作和 QSA 案例也沿用各自的历史提交。

8.3 Apple、Hygon、AMD：不同结论等级

Apple 的 Metal 记录涵盖几台不同设备。M4 上两个固定的 FP32 行归约任务，相对各自的起点保留了 1.107 和 1.138 倍的历史改进；M1 Pro 上的小形状 GEMM+SiLU，相对朴素任务基线确认了 23.604 倍。后一数值主要说明起点留下了很大优化空间，所用基线与专家库实现不同。M2 的 GEMM 搜索虽有较低读数，计时稳定性尚不足以给出性能结论。Metal 的样本覆盖整个 command buffer，并按重复调用均摊；它和 NVIDIA 的 CUPTI 样本各有自己的区间 [21]。

Hygon BW1101 展示了另一种结果组合。逐通道矩任务从 9.439 降到 6.719 μ s，原记录判为确认加速；prelu 则从 5.439 增至 9.759 μ s，属于确认变慢。多项短内核的两边都读到 5.439 μ s，来源把它们标为测量分辨能力待查，而非真正持平。部分历史行没有在汇总表转录完整形状，读者可以沿链接回到原运行查看 [22]。AMD gfx1151 现有 Triton/HSACO 路线和设备调查；两种计时器的绝对时间仍需对齐，因此其结果页暂不列加速比 [23]。

表 8 Apple、Hygon 与 AMD 的代表性观察。每行沿用自己的任务基线和计时资格。

目标	固定案例	观察	可陈述的范围
Apple M4	两项 FP32 行归约	1.107 / 1.138 倍	各自任务的历史晋升记录
Apple M1 Pro	GEMM+SiLU，小形状	23.604 倍	相对朴素任务基线，非专家库对照
Hygon BW1101	逐通道矩	1.405 倍	首个保留合格 Run 的本机结果
Hygon BW1101	prelu	0.557 倍	确认变慢，应保留负结果
AMD gfx1151	RMSNorm smoke	延迟缺失	路线调查；计时器未对齐

8.4 MetaX：单内核、完整输出和实际 Run 分别验收

C550 的记录覆盖三件互有关联的工作。第一项是固定形状 M17 GEMM 的单内核分块优化，下节详述它相对本机基线的独立确认。第二项是 GQA、MLA 和 MoE 等 17 个程序变体的完整输出核对，86 个 case 全部通过；这些收据记录的是正确性，没有普通程序计时样本。第三项是一次十轮 Agent Run，保留了候选被拒、修正和最终确认的过程。该次 Run 中 12.032/11.008 μ s 的配对中位数属于固定基线与 local_serialized 运行环境。三种观察各自回答实现、判对和作者流程的问题，详见 C550 专题与集成案例。

8.5 案例 A : C550 的局部 tiling 收益

1.241 倍 C550 · FP16 GEMM · $M = 17, N = 128, K = 2048$

基线 $72.448 \mu\text{s}$ → 候选 $58.368 \mu\text{s}$; M tile $64 \rightarrow 32$ 。独立确认的正确性与测量质量门通过。

这是 `local_serialized` 范围内的本机 authoring 对照，支持这个形状上的局部调度改进。这次只改变 M tile，独立确认说明更小的 tile 对当前形状有利。将这一观察提炼为跨目标经验，还需要在目标设备上重新选择参数并验收收益 [24]。

8.6 案例 B : B300 的对齐条件与质量门

1.081 倍 B300 · FlashInfer 任务 026 · 守卫式对齐

固定 generic sliced-w8 对照 $6.4005 \mu\text{s}$ → 候选 $5.920 \mu\text{s}$ ；同次候选对照边通过计时质量门。

这项对齐改动对固定的通用实现取得了可确认的局部收益；与外部参考的那组样本没有通过 CV 质量门，因此外部胜负仍未确定。任务 008 则把两阶段 Program 跑通并通过 2550 份数值快照，但候选对内部对照和外部参考的计时都不稳定。它提供了完整程序的正确性记录，性能判断需要新的合格样本 [20]。

8.7 两个局部案例的结论范围

两个案例分别说明了分块与对齐可以在固定任务上改进本机候选。它们使用不同硬件、任务和计时协议，因此正文按案例报告，同时保留变慢、差异不足和计时失败的记录。当前 [状态页](#) 也没有汇总全项目已验收的 Study Report；各项结果继续从自己的报告和收据追溯。

8.8 案例 C : B300 MoE 的事件流水与 native CUDA lowering

2026-09-28 后续案例 B300-M4 最多四卡

固定的四 rank BF16 MoE 层中，N128 候选在四路由 $\times K = 4/2/1$ 的 12 个控制上有 **0/50,331,648** 个 FP64-oracle 失败、**960/960** 个 CPU/GPU tile-event 计数一致；12 个控制均记录到计算任务与下一事件生产重叠。此处是有界正确性和执行证据，不是对开源实现的加速声明。

这一增补的实验代码单独固定于尚未合入主线的 2c638aad，作业编号、外部证据根与结论边界见 [MoE 案例记录](#)。RankedTileEffects.analyze 由 ProgramMap 推导各阶段的 CTA 工作单元与安全任务容量；lower_ranked_tiles 把 up/gate、activation、down 的 Cake Schedule 与事件发布、前驱依赖和有界偷取绑定为一个完整的 B300 worker。一个 rank 的一层只启动一次持久 cooperative grid：前 c 个 CTA 生产路由、Bin 和 tile，其余 CTA 消费就绪任务；生产者完成事件工作后可按预算领取剩余任务。时间重叠通过任务完成计数在下一事件生产期间增长来核对，并非仅依据两段时间窗口相交推断。

优化不是一次性换掉后端，而是按失败证据修改 lowering。最初每事件一次 launch 变成每层一次，样本中的 layer kernel 数由 92 降至 16，却未得到整体延迟收益。下一事件 lookahead 与双角色流水建立了实际任务重叠，早期完整层计时仍接近持平。随后把同 GPU 生产者 barrier 的 PTX acquire/release 收窄为 device scope，跨 GPU 发布仍保留 system scope；远端 BF16 payload 在对齐时使用 128-bit PTX 加载/存储，非对齐子视图走标量路径。emit_tensor_tile_stage 进一步把

activation 从每 CTA 一行改为四行、六个 warp 各一行，并给最后的不足六行加掩码；N128 版本同时改变 TMA box、MMA 形状、TMEM 列数、共享内存位置与输出偏移。这些资源和任务容量随 Schedule 派生，不由 Evaluation 另行指定。

表 9 MoE 案例中逐步改变 lowering 的开发性观察。各行的工作负载或对照版本不同，比值不能相乘，也不构成对外部基线的速度比较。

改动	lowering 中改变的承诺	有界观察
持久事件网格	每事件 launch → 每 rank 每层一个网格	92 → 16 个 layer kernel；未加速
作用域与通信	GPU-scope 生产者 barrier；对齐时向量化 peer payload	两次 barrier 对照 1.049–1.116 倍；通信较重路由 1.016–1.019 倍
activation 工作单位	每 CTA 一行 → 四行 → 六 warp 六行	一行到四行 1.294–1.379 倍；四行到六行 1.010–1.045 倍
N128 tensor tile	每 tile 78 → 50 个阶段任务；TMEM 64 → 128 列	同租约 N64/N128 中位数 6.344/6.091 ms，1.042 倍

N128 还通过六组空间/偷取控制：0/25,165,824 个 oracle 失败、480/480 个事件计数一致，零预算没有偷取，开放预算发生实际偷取且未越界；六组均记录到时间重叠。首个 N64/N128 ABBA 的末尾 N64 arm 曾异常达到 27.082 ms，因此其自动汇总比值没有被采纳；上述 1.042 倍来自独立反向顺序实验，仍只覆盖合成 fanin、 $K = 4, c = 64$ 的开发性 CUPTI 比较。SGLang/DeepEP 开源基线已与 Cake 核对到完全相同的输入和 FP64 oracle，但两者单次 Nsight 记录分别为 2.149 和 7.335 ms，来自不同租约与 profiler 版本。当前不能声称相对该基线加速； $c = 95$ 的间歇性 route-location Finding 也仍开放 [26]。

8.9 历史 B200 案例的放置方式

2026 年 9 月旧版报告 还保存了两项 B200 案例。QSA 的 $T = 32768$ 完整程序，历史确认候选和同次 CUDA 黑盒参考分别约为 127.8 与 676.4ms，展示了多阶段计划与程序级评测的一次实际使用。Flash-KMeans 的固定形状配对搜索，每组有三次独立运行；Open-Cake 与直接 CUDA 编写环境的确认终点中位数分别约为 1.512 和 5.342 ms。这项本地实验采用 150k token 和 max 努力级别；CAKE 论文采用另一套 80M-token 协议，第 8 节将并列说明条件。两项结果都作为历史快照引用，完整任务、软件和样本来源列在附录 [18]。

9 与 CAKE 对照：相同的问题，不同的证据

本仓库与 CAKE 有共同的研究问题，也有可以直接检查的重叠任务。不过，论文聚合数字、公开的 CAKE 生成产物、本仓库运行记录和某个任务的外部参考，各由不同材料产生。表10先比较研究设计与实现范围；下节再看同一 TinyGEMM2 任务上能够进行的数值和速度对照 [1, 18]。

CAKE 论文表 2 以 B200 上固定的 FlashML 基线 (0.938 ms) 为参照，在 80M token 截点报告 Cake IR 三次运行最优候选的中位数为基线的 1.144 倍，直接 CUDA/PTX 为 0.928 倍。这里转录的是论文结果。本仓库历史 Flash-KMeans 实验使用 150k token、不同努力级别和自己的基线，因此表10并列的是协议，而不是用两个实验的加速比互相比大小 [1, 18]。

表 10 CAKE 论文与本仓库在研究问题、目标和已获得证据上的对应关系。论文数字与本地运行分别标明来源。

问题	CAKE 论文 v1	本仓库当前可读证据
执行计划	硬件显式 Cake IR; NVIDIA Ampere–Blackwell	独立 typed Schedule/Program; 五 Vendor、八精确 Target 文档
静态反馈	Verifier 四类门与目标校准成本排序	共享四类诊断和后端预检; 无目标校准时报告覆盖限制
作者处理	B200、固定模型和 scaffold, Cake 对直接 CUDA/PTX, 3+3 次, 80M token	历史 B200 配对运行有自己的 150k/max 条件; 当前无可与论文对齐的 clean-start 估计
专家端点	KDA、TinyGEMM、Alpha-MoE 等生产任务及服务/portfolio 结果	部分固定任务复现与完整输出; 大范围 dispatcher、服务及部分原语仍缺
非 NVIDIA	原论文未测目标侧 lowering 与成本	Apple、AMD、Hygon、MetaX 逐目标实现; 正确性和计时资格不均

9.1 TinyGEMM2 : 相同任务上的性能差距

TinyGEMM2 提供了更具体的对照。仓库固定三个 B300 BF16+bias 形状，用独立 FP64 数学参考核对结果，再要求候选与官方导出产物逐位一致。本地候选的 30 次检查和官方导出的 15 次检查全部通过。随后，五轮交替顺序的 CUPTI 冷 L2 测量保留了两组完整样本。表11列出 stage4 实现的中位数：本地候选已经重现了任务的数值结果，速度仍落后于对应的 CAKE 导出产物[25]。

表 11 B300 三个固定 TinyGEMM2 形状的历史中位数 (μs)。减速倍数使用原始成对样本计算；此表比较生成产物，不是论文的智能体起点实验。

B/N/K	CAKE 导出 stage4	本地 stage4	本地相对减速
1/128/720	2.720	100.192	36.77 倍
16/1024/1024	3.040	105.825	34.81 倍
64/4096/3072	21.216	289.186	13.63 倍

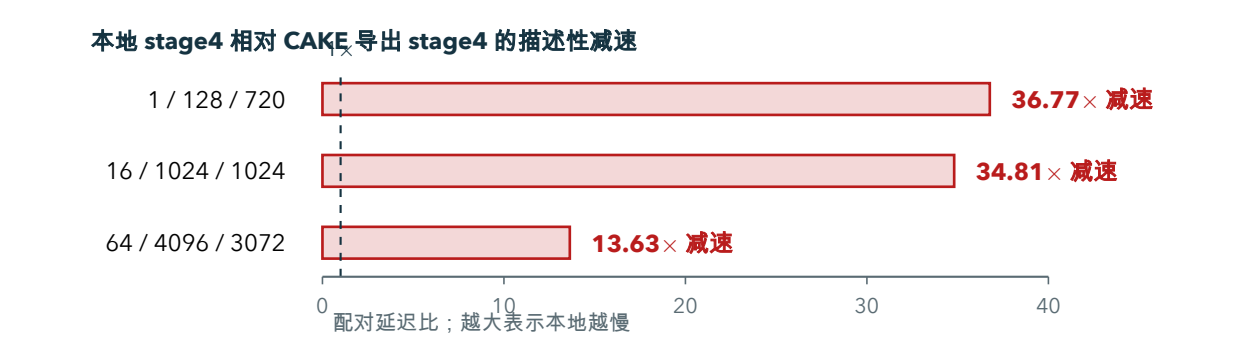


图 7 严格数值检查通过后，本地 stage4 在三个 B300 固定形状上仍比官方 CAKE 导出 stage4 慢。图示历史配对中位数；各轮原始样本与质量记录见 专题。

差距提示下一步要检查 TMA、角色流水线和资源使用，但现有 profiler 记录尚不足以把 13–37 倍的减速归给其中某一个原因。CAKE 导出产物原有的更大形状集合与服务验证，以及 KDA、Alpha-MoE 的完整任务，也仍在本仓库后续复现范围之外[25]。

10 跨硬件复用：从一次改写到可检验的收益

10.1 可复用的是机制，参数要在目标卡重选

一次优化既可能发现可复用的做法，也会得到只适合当前设备的参数。比如，融合私有的中间张量可以省去一次写回和重读；能否融合取决于消费者、生命周期与舍入语义。具体的 tile、执行组和流水线深度，则要在目标卡上重新选择。可确定性表达的改写可以成为带前提和拒绝原因的显式 pass；只有一次有效的局部参数选择，仍作为任务经验保留 [27]。

10.2 E/P 四组实验

若要知道经验究竟提供了多少帮助，先要把“让智能体读说明”和“让智能体调用现成改写”分开。图8以 E 控制额外机制材料，以 P 控制显式 pass 的调用权，形成四种条件。四组共享任务、编译器、目标设备、正确性和计时规则；工具权限由运行环境实际隔离。 $P1$ 组仍需知道调用的最小接口，所以 E 测量的是接口之外的说明和案例。所引主体快照提供软件协议；本文不由其直接报告四组处理效果 [28]。后继实验须按第2节的共同主题及各自冻结的 Study 解释。



图8 四组迁移实验的处理条件。 E 是额外说明与案例， P 是显式变换调用权；所有组使用相同的编译器和设备评测。此图表示实验分组，尚无实机处理效果。

令 p_{ep} 为在预注册预算内经新鲜确认、达到本机预定改善阈值的 Run 比例。预定主要对比为

$$\begin{aligned}\Delta_E &= \frac{1}{2}[(p_{10} - p_{00}) + (p_{11} - p_{01})], \\ \Delta_P &= \frac{1}{2}[(p_{01} - p_{00}) + (p_{11} - p_{10})], \\ \Delta_{EP} &= p_{11} - p_{10} - p_{01} + p_{00}.\end{aligned}$$

一次独立运行是重复单位。正式研究按任务等权汇总，保留各任务内四组的配对关系；搜索成本、预算耗尽和基础设施缺失也按事先约定记录。四组全部运行并经共同确认后，才能计算上面三个处理效果。

10.3 从 B300 到 Hygon：可执行与性能的分界

迁移专题记录了一次具体尝试。B300 上的 pairwise_sqdist 候选把 K 维按 256 分块；在固定 FP32 形状上，五类输入检查和 CUPTI 配对计时通过，相对本机基线从 411.7945 降到 53.856 μ s。将同一数学任务准备给 Hygon BW1101 时，团队保留张量、输入、参考答案和容差，先把旧版 Schedule 的角色表示显式适配到当前 execution_groups。适配后的候选及同卡起点均通过编译器检查，并由 Hygon Triton 编成 HSACO。

原 bw1100 节点的八卡被其他 GLM 进程占用，实验止于离线准入。另一台 bw1100-1 具有空闲 BW1101，但宿主内核和环境与前者不同；因此先为该节点单独捕获 Executor，再在干净源码

4c9f4cc0 上运行相同合同。五类输入在计时前后均判对；新鲜 HIP 配对确认质量通过，同卡起点/来源 $K=256$ 候选为 $517.9645/748.749 \mu s$ ，十对均是起点更快。故该机制能够正确执行，原参数却在此节点产生负迁移。本地串行锁不能排除没有使用同一锁的外部任务；不同 CUPTI/HIP 计时器的绝对时延也不构成跨卡速度比。

在运行下一次设备实验前，冻结两个只改 K tile 的 Hygon 后继：128 和 64。离线编译均报告零 scratch，分别占 252 和 148 个 VGPR；按预定的资源排序只选择 $K=64$ 上卡。它相对 $K=256$ 的生成源码仅改变分块值与调度身份。在相同五类输入、whole- K 起点及 HIP 配对协议下，新鲜独立确认全部判对并通过计时质量门，起点/候选为 $518.524/189.586 \mu s$ (2.735 倍)，十对均是候选更快； $K=128$ 没有设备计时。因此， K 维分块这一来源机制可以在目标卡重新选参后奏效，但来源参数不可照搬。另在上卡前冻结一个未见的同算子形状 $R=1024, K=512, N=64$ ，只比较已选 $K=64$ 与该形状的 canonical whole- K 起点，不再试其他 tile。五类输入在首次及新鲜独立确认中均零不匹配、计时质量通过；确认中位数为 $106.232/97.593 \mu s$ (1.0885 倍)，十对均为候选更快。第二个形状提供有限的 Hygon 本机形状泛化证据，但收益远小于 $K=1024$ 的 2.735 倍；B300 没有在新形状上的来源性能收据，也没有跨算子结论。这是一例限定形状的人工机制迁移和 Hygon 本机收益，并非控制作者与预算、随机分配材料 E 或变换权限 P 后得到的 Agent 处理效果。

难点也并非单一后端开关：同一带循环归约的 Schedule 在当前 Metal 路线无法表达；Triton 能向 AMD、Hygon 生成代码，却不保证源参数的资源分配或速度；同 ISA 的第二台 Hygon 宿主仍需独立 Executor；C550 的容器锁边界和 M2 的计时质量分别限制设备主张。可表达性、目标选参、宿主身份、测量与因果归因须分层验收。

11 限制、可复查性与结论

Open-Cake-IR 已经把候选生成、静态检查、目标代码生成和设备反馈接成一条可追踪的路径。在主体快照源码上，179 项语料符合预期；在若干固定 Workload 上，也已有设备正确性与局部性能结果。这些结果回答了“系统能否表达、检查并运行具体候选”。要回答“这一机制能否稳定带来跨任务、跨平台收益”，还需要更完整的目标侧测量和预注册对照实验。

11.1 解释结果时的四项限制

与 CAKE 的关系。本文复现的是论文描述的设计问题与实验思路，并独立实现 Compiler。论文公布了系统机制和汇总结果，但缺少把原始编译器、完整 IR 与 Verifier、agent scaffold、clean-start 轨迹及计时细节一起重建的材料。因此，同名任务的对照应从输入合同、参考实现和测量范围逐项核对 [18]。

平台覆盖。五家厂商、八份 Target 文档界定了可声明的目标；每个平台还须单独取得工具链、数值、计时器和 profiler 的运行收据。179 项 Corpus Gate 尚未检查 apple_gpu_family7 与 apple_gpu_family9。MetaX 的完整输出验证和 AMD 的计时器调查，也分别处于不同的证据阶段 [19, 14]。

性能范围。一个形状的候选优于本机基线，并不意味着它胜过外部专家实现，或能直接用于多形状应用。后者还涉及分派守卫、尾部输入、回退路径和框架开销。历史 QSA 数字覆盖完整 Program 的 GPU 区间；M17 数字覆盖单内核。两者都不能直接换算成模型或服务延迟。

机制归因。工程 Run 和局部消融可以发现问题、改进系统。若要判断作者环境或迁移材料带来了多少增益，还须固定处理条件，进行独立 Run 和共同确认，并说明缺失结果如何计入分析。 E/P 四组的实机对照尚待开展。

11.2 下一步的可检验目标

后续实验按第2节统一问题和证据口径。先确保目标基线、完整输出、计时和 profiler 可用，再依次检验同动作集合的非 LLM 对照、原生 DSL 加等效工具的表示对照，以及来源经验与可执行化的增量。缺少合格低层路线时明确报告覆盖，不借其他目标的资格启动实验。反复出现的缺口通过 Finding、审查和后继提交处理；未参与机制开发的任务检验复用，完整 Program 和应用收益在强种子上单独验收。

A 术语、源码入口与完整示例

表 12 文中的术语按负责事实区分。英文名是 API 查找键，表中中文名用于叙事。

对象	本报告中的固定含义
Workload	外部算子语义、输入域、oracle 与数值容差
Schedule	单内核的操作、资源、执行组、访问与同步承诺
Program	公共张量、多个叶子 Schedule 的阶段顺序和绑定
Target	精确设备声明；事实未建模时不借相邻型号
Run	一次冻结的作者环境、候选、预算与终态
作者	在一次 Run 中提交候选的人或智能体；可见材料与工具由运行规格限定
Study	运行前固定研究处理、分配、估计量和分析规则
Finding	可定位、可归属的结构、目标或后端诊断

当前 [Softmax 文件](#) 是第 4 节运行示例的完整可读源码：

```
from open_cake_ir.compiler import frontend as cake

@cake.schedule(name="softmax-b8-smoke", target="sm_100a", backend="triton",
               entry_point="cake_softmax_b8_smoke",
               residency={"ctas_per_multiprocessor": 4,
                         "registers_per_thread": 96})
def softmax(lm, x: cake.Tensor((8, 512, 128), "fp32"),
            y: cake.Tensor((8, 512, 128), "fp32", mode="output")):
    compute = lm.role(execution_groups=[0, 1, 2, 3])
    row_block = lm.program(x, axis=0, dimension=1, tile=64)
    batch = lm.program(x, axis=1, dimension=0, tile=1)
    with compute:
        x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
        rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
        shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
        weights = lm.exp(shifted, id="exponentiate")
        rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
        y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
        lm.store(y[batch, row_block, :], y_tile, id="store_y")
```

这里的 Python 接口通过 AST 构造与 JSON 相同的规范计划；`Compiler.load()` 默认读取项目的当前配置。要在具体 GPU 上运行，还需匹配目标工具链，并完成正确性与性能验收。[Python 前端文档](#)列出了支持的语法及源码位置诊断。

B 结果来源与历史身份

表 13 正文各类结果的第一阅读入口。报告页面是带范围的投影，原始运行收据继续拥有测量与监管事实。

范围	固定来源或索引
当前 Compiler	状态页、版本清单与精确目标文档
B300 FlashInfer	逐任务状态；每行链接自己的 Finding 与收据
B300 TinyGEMM2	严格数值与成对计时报告
B300 MoE 增补	持久事件流水、N128 与开源基线案例；实验代码另固定于未合入主线的 2c638aad
Apple、Hygon、AMD	各平台 发布汇总 与原记录
MetaX C550	平台专题及 当前 Run 案例
迁移离线试点	机制与证据边界
历史案例	QSA 与 Flash-KMeans：2026-W36.1 旧报告及其证据附录；退役路径按 history 标签回放

历史 Flash-KMeans 的六次终点属于固定 $B = 32, N = 65536, K = 1024, D = 128$ 、BF16 输入的 B200 Workload，三对延迟（Open-Cake/CUDA，ms）为 1.191845/3.898164、1.511927/7.130795 和 1.512163/5.342292；两组各取自己的中位数，范围不是置信区间。历史 QSA 的 $T = 32768$ 完整 Program 候选和同次参考分别为 127.821446/676.423598 ms；该时间不含 Python 包装开销。这里只转录旧报告的有界结论与身份，不以新版 Compiler 重释旧原始记录。

C 复现边界与报告构建

本文的数字来自已存在的报告、Finding 与原始收据定位；写作和 TeX 编译没有启动新的 provider、GPU Campaign 或计时。阅读者可从固定源码检出运行离线入口，看到 IR 构造、Assessment 与 lowering；设备结果则须在原任务的目标、工具链、参考访问和监管环境下重放。报告源文件位于 docs/open-cake-ir-technical-report.tex，在装有 XeLaTeX、中文字体的环境中可直接编译；图与参考文献均保存在单个 TeX 源文件中。历史示意图保留其原始来源；其文字表达的能力范围按各章节明确的源码范围解释，不用旧图中的版本值替代当前状态。

阅读与引用说明

本文是项目维护者撰写的仓库技术报告，无 DOI、期刊或同行评审声明。引用设计时请固定本报告及其所读源码提交；引用性能时还需追溯案例自己的执行源码、精确目标、Workload、baseline、oracle、计时区间与质量门。历史实现章节的仓库链接固定到 677eb444；研究主题增补引用文档提交 bdc8e62。MoE 增补的主线案例记录固定到 93d05202，实验代码另固定到未合入主线的 2c638aad，原始收据身份仍由该专题继续给出。正式英文引文题名沿用仓库 CITATION.cff 的 preferred-citation；封面的中文题名概括本文的研究对象。

参考文献

[1] Z. Ye et al. *CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution*. arXiv:2608.12629v1, 2026. <https://arxiv.org/html/2608.12629v1>.

[2] NVIDIA. *CUDA Toolkit 12.1 Release Notes*. <https://docs.nvidia.com/cuda/archive/12.1.0/cuda-toolkit-release-notes/>.

- [3] NVIDIA. *CUDA Toolkit 12.2 Release Notes*. <https://docs.nvidia.com/cuda/archive/12.2.0/cuda-toolkit-release-notes/index.html>.
- [4] MLC-AI. *TIRx Harness*. <https://github.com/mlc-ai/TIRx-harness>.
- [5] Y. Ikarashi et al. *Exo 2: Growing a Scheduling Language*. <https://arxiv.org/abs/2411.07211v4>.
- [6] LLVM. *Transform Dialect*. <https://mlir.llvm.org/docs/Dialects/Transform/>.
- [7] T. Chen et al. *Learning to Optimize Tensor Programs*. <https://arxiv.org/abs/1805.08166>.
- [8] J. Shao et al. *Tensor Program Optimization with Probabilistic Programs*. <https://arxiv.org/abs/2205.13603>.
- [9] J. Du et al. *AKG kernel Agent*. <https://arxiv.org/abs/2512.23424v1>.
- [10] K. S. Dong et al. *KernelBlaster*. <https://arxiv.org/abs/2602.14293>.
- [11] G. Zhang et al. *AccelOpt*. <https://arxiv.org/abs/2511.15915>.
- [12] F. Costa et al. *AI as a Compiler: Compiling Triton kernels without the Triton compiler*. <https://arxiv.org/html/2609.36800v1>.
- [13] B. Driscoll et al. *Equivalence Checking of ML GPU Kernels*. <https://arxiv.org/abs/2511.12638v3>.
- [14] H. Qin. 系统架构与平台能力. open-cake-ir@677eb444, 2026.
- [15] H. Qin. IR 指南. open-cake-ir@677eb444, 2026.
- [16] H. Qin. Compiler 清单与 Target 文档示例. open-cake-ir@677eb444, 2026.
- [17] H. Qin. 验收门. open-cake-ir@677eb444, 2026.
- [18] H. Qin. 论文与本地证据合同. open-cake-ir@677eb444, 2026.
- [19] H. Qin. 当前状态. open-cake-ir@677eb444, 2026.
- [20] H. Qin. B300 FlashInfer 任务综述. open-cake-ir@677eb444, 2026.
- [21] H. Qin. Apple 平台结果. open-cake-ir@677eb444, 2026.
- [22] H. Qin. Hygon DCU 平台结果. open-cake-ir@677eb444, 2026.
- [23] H. Qin. AMD 平台结果. open-cake-ir@677eb444, 2026.
- [24] H. Qin. C550 专题与 M17 独立确认. open-cake-ir@677eb444, 2026.
- [25] H. Qin. B300 CAKE 重叠任务对照. open-cake-ir@677eb444, 2026.
- [26] H. Qin. B300 MoE 持久事件流水与 N128 验证. open-cake-ir@93d05202; 实验源码 2c638aad 尚未合入主线, 2026.
- [27] H. Qin. 跨硬件优化知识迁移机制. open-cake-ir@677eb444, 2026.
- [28] H. Qin. E/P 消融实验设计. open-cake-ir@677eb444, 2026.